

Machine Learning for Physicists

Day 1: Introduction and classifiers

Sascha Cassandra Diefenbacher, Heidelberg University

Sources. These are lecture notes for the first day of a four-day lecture series. Large parts are taken from, or closely follow, the lecture notes *Modern Machine Learning for LHC Physicists* by T. Plehn, A. Butter, B. Dillon, T. Heimel, C. Krause, and R. Winterhalder, Ref. [1], whose notation we mostly follow.

- Passages taken from Ref. [1] are marked with a grey bar and the section they are taken from. Omissions are marked by [...]; notation and cross-references are adapted to these notes.
- Passages which follow Ref. [1] but are rewritten are marked as *adapted*, again with the section.
- Where we deviate from Ref. [1] to correct an error, this is stated in a footnote.
- Figures from other publications are cited in their captions.

We quote and cite the updated version of Ref. [1] from June 2026, available at https://www.thphys.uni-heidelberg.de/~plehn/pics/modern_ml.pdf; its section numbers and wording differ in parts from the arXiv versions. Ref. [1] is distributed under the Creative Commons Attribution 4.0 International license (CC BY 4.0). The gradient-descent example of Sec. 2 follows the author's introductory lecture at the ML4FP school 2025.

Use of generative AI. These notes were prepared with the help of a generative AI assistant (Claude, Anthropic). It was used to compile the passages from the sources above, to draft parts of the remaining text and the step-by-step derivations, and to write the code for the figures without a cited source. All content was reviewed and edited by the author, who is responsible for any remaining errors.

1 Models as numerically defined functions

From a practical perspective, we start with the assumption or observation that neural network are nothing but numerically defined functions. Some kind of minimization algorithm on a loss function will allow us to determine its parameters θ . The simplest case, regression networks, are scalar or vector fields over some feature or phase space, $f_\theta(x)$. Assuming that we have access to the true target values y in form of a training dataset $(x, y)_j$, we want to construct the approximation

$$f_\theta(x) \approx y . \quad (1.1)$$

[adapted from Ref. [1], Sec. 1.2.2, *Fits and interpolations*, target called y]

In physics, x is usually some kind of phase space, and we can distinguish four kinds of functions over phase space:

1. regression targets or likelihoods $f_\theta(x)$, today and on Day 4;
2. likelihood ratios $p_1(x)/p_2(x)$ as optimal classifiers, today and on Day 3;
3. probabilities we can sample from $x \sim p(x)$, on Day 2;
4. conditional probabilities $x \sim p(x|y)$, on Day 2.

[adapted from Ref. [1], Sec. 1.2.3, *Neural networks*, references to this series]

Whatever the task, the model is a function with inputs x , adjustable parameters θ , and an output we want to be close to a desired target. Everything else in this lecture series is about (i) which functions f_θ we use, (ii) how we measure closeness, and (iii) how we adjust θ .

2 Fitting a model: gradient descent

We start with the simplest possible example, one data point $(x, y) = (1, 1)$ and the constant model

$$f_\theta(x) = a \quad \text{with} \quad \theta = \{a\} \quad \text{and initial value} \quad a_0 = 2 . \quad (2.1)$$

To quantify how close the model is to the data we define a loss function, here the squared difference,

$$\mathcal{L}(a) = (f_\theta(x) - y)^2 = (a - 1)^2 \quad \Rightarrow \quad \mathcal{L}(a_0) = (2 - 1)^2 = 1. \quad (2.2)$$

Its derivative follows from the chain rule,

$$\frac{\partial \mathcal{L}}{\partial a} = 2(f_\theta(x) - y) \frac{\partial f_\theta(x)}{\partial a} = 2(a - 1) \cdot 1 \quad \Rightarrow \quad \left. \frac{\partial \mathcal{L}}{\partial a} \right|_{a_0} = 2(2 - 1) = 2. \quad (2.3)$$

The derivative is positive, so $\mathcal{L}$ grows with a , and we move a in the opposite direction,

$$a_1 = a_0 - \left. \frac{\partial \mathcal{L}}{\partial a} \right|_{a_0} = 2 - 2 = 0 \quad \Rightarrow \quad \mathcal{L}(a_1) = (0 - 1)^2 = 1. \quad (2.4)$$

We have overshoot the target $a = 1$ and ended up just as far away on the other side. The next step would bring us back, $a_2 = 0 - 2(0 - 1) = 2$, and we would oscillate forever. The fix is to only take a fraction η of the step, the learning rate,

$$a_{t+1} = a_t - \eta \left. \frac{\partial \mathcal{L}}{\partial a} \right|_{a_t} \xrightarrow{\eta=1/2} a_1 = 2 - \frac{1}{2} \cdot 2(2 - 1) = 2 - 1 = 1 \quad \Rightarrow \quad \mathcal{L}(a_1) = 0. \quad (2.5)$$

For this example we can follow the update for any learning rate. Subtracting the target value from both sides of the update rule gives

$$a_{t+1} - 1 = (a_t - 1) - \eta \cdot 2(a_t - 1) = (1 - 2\eta)(a_t - 1) \quad \Rightarrow \quad a_t - 1 = (1 - 2\eta)^t (a_0 - 1). \quad (2.6)$$

The distance to the minimum shrinks for $|1 - 2\eta| < 1$, that is $0 < \eta < 1$. For $\eta < 1/2$ we approach the minimum from one side, for $1/2 < \eta < 1$ we overshoot in every step but still converge, $\eta = 1/2$ reaches the minimum in one step, $\eta = 1$ oscillates as above, and $\eta > 1$ diverges. This is gradient descent: repeat the step until the loss is small enough, and keep η small enough to not overshoot.

Exercise (in class): three points $(1, 1)$, $(2, 2)$, $(3, 2)$, the linear model $f_\theta(x) = ax + b$ with $a_0 = b_0 = 1$, and the mean squared error (MSE), the mean of the squared differences between model and data,

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (f_\theta(x_i) - y_i)^2. \quad (2.7)$$

Compute $\partial \mathcal{L} / \partial a$ and $\partial \mathcal{L} / \partial b$ and perform one step with $\eta = 0.1$.

Solution. With $a_0 = b_0 = 1$ the model predicts $f_\theta(x_i) = x_i + 1 = (2, 3, 4)$, so the residuals $r_i = f_\theta(x_i) - y_i$ and the loss are

$$r_i = (1, 1, 2) \quad \Rightarrow \quad \mathcal{L}_0 = \frac{1}{3} (1^2 + 1^2 + 2^2) = \frac{6}{3} = 2. \quad (2.8)$$

With $\partial f_\theta(x_i) / \partial a = x_i$ and $\partial f_\theta(x_i) / \partial b = 1$ the chain rule gives

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial a} &= \frac{2}{N} \sum_i r_i x_i = \frac{2}{3} (1 \cdot 1 + 1 \cdot 2 + 2 \cdot 3) = \frac{2}{3} \cdot 9 = 6, \\ \frac{\partial \mathcal{L}}{\partial b} &= \frac{2}{N} \sum_i r_i = \frac{2}{3} (1 + 1 + 2) = \frac{8}{3} \approx 2.67. \end{aligned} \quad (2.9)$$

The update with $\eta = 0.1$ is

$$a_1 = 1 - 0.1 \cdot 6 = 0.4 \quad \text{and} \quad b_1 = 1 - 0.1 \cdot \frac{8}{3} = \frac{11}{15} \approx 0.733. \quad (2.10)$$

The new model predicts $f_\theta(x_i) = 0.4x_i + 11/15 = (17/15, 23/15, 29/15) \approx (1.13, 1.53, 1.93)$, so

$$r_i = \left(\frac{2}{15}, -\frac{7}{15}, -\frac{1}{15} \right) \quad \Rightarrow \quad \mathcal{L}_1 = \frac{1}{3} \cdot \frac{4 + 49 + 1}{225} = \frac{54}{675} = \frac{2}{25} = 0.08. \quad (2.11)$$

One step reduces the loss from 2 to 0.08. Where does gradient descent end? At the minimum both derivatives vanish, which gives the normal equations of a linear fit, with $\sum x_i = 6$, $\sum x_i^2 = 14$, $\sum y_i = 5$, $\sum x_i y_i = 11$,

$$\begin{aligned} \sum_i (ax_i + b - y_i) x_i &= 0 \Leftrightarrow 14a + 6b = 11, \\ \sum_i (ax_i + b - y_i) &= 0 \Leftrightarrow 6a + 3b = 5. \end{aligned} \quad (2.12)$$

From the second line $b = (5 - 6a)/3$, inserted into the first $14a + 2(5 - 6a) = 11$, so $2a = 1$ and

$$a = \frac{1}{2}, \quad b = \frac{2}{3}, \quad r_i = \left(\frac{1}{6}, -\frac{1}{3}, \frac{1}{6}\right), \quad \mathcal{L}_{\min} = \frac{1}{3} \left(\frac{1}{36} + \frac{1}{9} + \frac{1}{36}\right) = \frac{1}{18} \approx 0.056. \quad (2.13)$$

The loss does not reach zero, because no straight line goes through all three points. The hands-on notebook reproduces all these numbers.

For a network with many parameters θ_j the step is the same for each parameter,

$$\theta_j^{t+1} = \theta_j^t - \alpha \left\langle \frac{\partial \mathcal{L}^t}{\partial \theta_j} \right\rangle. \quad (2.14)$$

The minus sign means that our optimization moves against the direction of the gradient, and α is the learning rate. We could evaluate the expectation value over the full training dataset. However, especially for large datasets, we evaluate it over a small, randomly chosen subset of the training data,

$$\left\langle \frac{\partial \mathcal{L}}{\partial \theta_j} \right\rangle \approx \frac{1}{n_{\text{batch}}} \sum_{i \in \text{batch}} \frac{\partial \mathcal{L}_i}{\partial \theta_j}, \quad (2.15)$$

where $\mathcal{L}_i$ is the loss of the single training point i . This method is called stochastic gradient descent, and the subsets of the training data are called minibatches. We follow the progress of the training and the numerical value of the loss as a function of epochs, defined as the number of batch trainings required for the network to evaluate the full training sample.

[adapted from Ref. [1], Sec. 1.2.3, *Neural networks*]

In practice we use the Adam optimizer, an adaptive version of Eq.(2.14) with momentum.

3 Neural networks

Gradient descent only needs one thing from the model: that we can differentiate the loss with respect to its parameters. It does not care whether the model is a straight line or something much more complicated, so we can optimize any function we can differentiate. What we need is a way to build differentiable functions which are expressive enough for any problem we want to describe. Neural networks are exactly such a construction.

The key is to use building blocks which we can put together such that simple functions require a small number of building blocks and model parameters, and complex functions require many more of the same building blocks. We start by defining so-called layers, which in a fully connected or dense network transfer information from all D entries of the vector x defining one layer to all vector entries of the neighboring layers,

$$x \rightarrow x^{(1)} \rightarrow x^{(2)} \dots \rightarrow x^{(N)} \equiv f_{\theta}(x). \quad (3.1)$$

Our network consists of N layers, including one input layer x , one output layer $f_{\theta}(x)$, and $N - 2$ hidden layers. If a vector entry $x_i^{(n)}$ collects information from all $x_j^{(n-1)}$, we can make an ansatz for each step of this chain as

$$x^{(n-1)} \rightarrow x^{(n)} = W^{(n)} x^{(n-1)} + b^{(n)}, \quad (3.2)$$

where the $D \times D$ matrix W is referred to as the network weights and the D -dimensional vector b as the bias. [...] In our simple regression case we already know that over the layers we need to reduce the width of the network from the input vector dimension D to the output scalar $x^{(N)} = f_{\theta}(x)$.

— taken from Ref. [1], Sec. 1.2.3, *Neural networks*

In components, every node i of layer n collects all entries of the previous layer,

$$x_i^{(n)} = \sum_j W_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)}, \quad (3.3)$$

so a node with D inputs has $D + 1$ parameters. The linear model $ax + b$ from Sec. 2 is exactly one such layer, with one input and one node.

Affine transformations form a group, so combining affine layers always gives us an affine transformation, encoded in a more complicated manner. This means our network defined by Eq.(3.2) can only describe linear functions, albeit in high-dimensional spaces. To describe general functions, we need to introduce a non-linearity in our network. The simplest implementation is a so-called activation function for each node. Probably the simplest 1-dimensional choice is the so-called rectified linear unit

$$\text{ReLU}(x_j) = \max(0, x_j) = \begin{cases} 0 & x_j \leq 0 \\ x_j & x_j > 0 \end{cases}, \quad (3.4)$$

giving us instead of Eq.(3.3)

$$x^{(n-1)} \rightarrow x^{(n)} = \text{ReLU} \left[W^{(n)} x^{(n-1)} + b^{(n)} \right]. \quad (3.5)$$

Here we write the ReLU transformation of a vector as the vector of ReLU-transformed elements. [...] But a sufficiently deep network can now describe general, non-linear functions, and combining layers adds complexity, new parameters, and expressivity to our network function $f_\theta(x)$. There are many alternatives to ReLU as the source of non-linearity, and depending on our problem they might be helpful, for example by providing a finite gradient over the x -range.

— taken from Ref. [1], Sec. 1.2.3, *Neural networks*

That two affine layers are again one affine layer can be seen explicitly,

$$x^{(2)} = W^{(2)} \left(W^{(1)} x + b^{(1)} \right) + b^{(2)} = \underbrace{\left(W^{(2)} W^{(1)} \right)}_{\equiv W'} x + \underbrace{\left(W^{(2)} b^{(1)} + b^{(2)} \right)}_{\equiv b'}, \quad (3.6)$$

a single affine layer with weights W' and bias b' . Historically, the first choice was a step function, the perceptron. Its derivative vanishes everywhere except at the step, so it cannot be trained with gradient descent. ReLU is non-linear, fast to evaluate, and has the simple derivative

$$\frac{d\text{ReLU}(z)}{dz} = \Theta(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases}. \quad (3.7)$$

To train the network we need $\partial \mathcal{L} / \partial \theta_j$ for all weights and biases.¹ We write each layer in two steps, the affine part $z^{(n)}$ and the activation,

$$z^{(n)} = W^{(n)} x^{(n-1)} + b^{(n)} \quad \text{and} \quad x^{(n)} = \text{ReLU}(z^{(n)}), \quad (3.8)$$

with a single output $f_\theta = x_1^{(N)}$ and, for one training point, the loss $\mathcal{L} = (y - f_\theta)^2$. For a weight in the last layer the chain rule gives three factors,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_{1j}^{(N)}} &= \frac{\partial \mathcal{L}}{\partial f_\theta} \frac{\partial f_\theta}{\partial z_1^{(N)}} \frac{\partial z_1^{(N)}}{\partial W_{1j}^{(N)}} \\ &= \left[-2(y - f_\theta) \right] \cdot \Theta(z_1^{(N)}) \cdot x_j^{(N-1)} \equiv \delta_1^{(N)} x_j^{(N-1)}, \end{aligned} \quad (3.9)$$

¹Our derivation follows Ref. [1], Sec. 1.2.3, where the result for the last layer is written as $-2\sqrt{\mathcal{L}} x_j^{(N-1)}$. That form drops the sign of $(y - f_\theta)$ and the derivative of the activation function, which we keep here.

where we use $\partial z_1^{(N)}/\partial W_{1j}^{(N)} = \partial(\sum_k W_{1k}^{(N)} x_k^{(N-1)} + b_1^{(N)})/\partial W_{1j}^{(N)} = x_j^{(N-1)}$ and define $\delta_1^{(N)} \equiv \partial\mathcal{L}/\partial z_1^{(N)}$. For the bias the last factor is one, $\partial\mathcal{L}/\partial b_1^{(N)} = \delta_1^{(N)}$. One layer earlier, the weight $W_{ij}^{(N-1)}$ enters through $z_i^{(N-1)}$, which enters $z_1^{(N)}$ through $x_i^{(N-1)}$,

$$\begin{aligned} \frac{\partial\mathcal{L}}{\partial W_{ij}^{(N-1)}} &= \frac{\partial\mathcal{L}}{\partial z_1^{(N)}} \frac{\partial z_1^{(N)}}{\partial x_i^{(N-1)}} \frac{\partial x_i^{(N-1)}}{\partial z_i^{(N-1)}} \frac{\partial z_i^{(N-1)}}{\partial W_{ij}^{(N-1)}} \\ &= \delta_1^{(N)} \cdot W_{1i}^{(N)} \cdot \Theta(z_i^{(N-1)}) \cdot x_j^{(N-2)} \equiv \delta_i^{(N-1)} x_j^{(N-2)}. \end{aligned} \quad (3.10)$$

The pattern repeats for every layer. With several nodes in layer n each node of layer $n-1$ feeds all of them, so we sum over them, and the general rules are

$$\delta_i^{(n-1)} = \Theta(z_i^{(n-1)}) \sum_k W_{ki}^{(n)} \delta_k^{(n)}, \quad \frac{\partial\mathcal{L}}{\partial W_{ij}^{(n)}} = \delta_i^{(n)} x_j^{(n-1)}, \quad \frac{\partial\mathcal{L}}{\partial b_i^{(n)}} = \delta_i^{(n)}. \quad (3.11)$$

We first evaluate the network forward, Eq.(3.1), storing all $z^{(n)}$ and $x^{(n)}$, then compute the $\delta^{(n)}$ backwards from the output, reusing each one for the layer before. This back-propagation costs about as much as one forward pass, and it is what PyTorch does automatically in the hands-on session. [adapted from Ref. [1], Sec. 1.2.3, *Neural networks*]

To check that the network generalizes,

[...] we usually divide a ML-related dataset into three parts. The main part is the training data, between 60% and 80% of the data. The above-mentioned test data is then 10% to 20% of the complete dataset, and we use it to check how the network generalizes to unseen data, test for over-training, or measure the network performance. The validation data can be used to re-train the network, optimize the architecture or the different settings of the network. The crucial aspect is that the test data is completely independent of the network training.

— taken from Ref. [1], Sec. 1.2.3, *Neural networks*

If the loss on the test data is much larger than on the training data, the network has learned the statistical fluctuations of the training data, called over-training. [adapted from Ref. [1], Sec. 1.2.2, *Fits and interpolations*]

4 Loss functions from likelihoods

The loss is a differentiable performance metric: it has to measure what we want the model to do, and it has to have gradients with respect to θ . Accuracy, for instance, is a good metric but a bad loss, because it is piecewise constant in θ . The MSE we have used so far is not arbitrary, it follows from a likelihood.

We look for the parameters θ which are most probable given the training data, the pairs (x_j, y_j) which we collectively call x_{train} . We typically cannot evaluate this posterior $p(\theta|x_{\text{train}})$ directly, so we use Bayes' theorem to rewrite it,

$$\begin{aligned} \hat{\theta} &= \operatorname{argmax}_{\theta} p(\theta|x_{\text{train}}) = \operatorname{argmax}_{\theta} \frac{p(x_{\text{train}}|\theta) p(\theta)}{p(x_{\text{train}})} \\ &= \operatorname{argmax}_{\theta} \left[p(x_{\text{train}}|\theta) p(\theta) \right] \simeq \operatorname{argmax}_{\theta} p(x_{\text{train}}|\theta). \end{aligned} \quad (4.1)$$

The two steps in the second line are of a different kind. The evidence $p(x_{\text{train}})$ does not depend on θ , so dropping it does not change the position of the maximum, this step is exact. Dropping the prior $p(\theta)$ is an assumption: we take it to be flat, so that it does not affect our maximization. What remains is the likelihood $p(x_{\text{train}}|\theta)$, the probability of the training data for given parameters. We will meet the prior again on Day 4, as a regularization of the network parameters.

[adapted from Ref. [1], Sec. 1.2.2, *Fits and interpolations* and Sec. 1.2.4, *Likelihood loss*, with the steps after Bayes' theorem separated]

To maximize the likelihood we need to assume a functional form. The training points are independent, so the likelihood is a product over them. Each training point is a pair, and only the target depends on the model,

$$p(x_{\text{train}}|\theta) = \prod_j p(x_j, y_j|\theta) = \prod_j p(y_j|x_j, \theta) p(x_j). \quad (4.2)$$

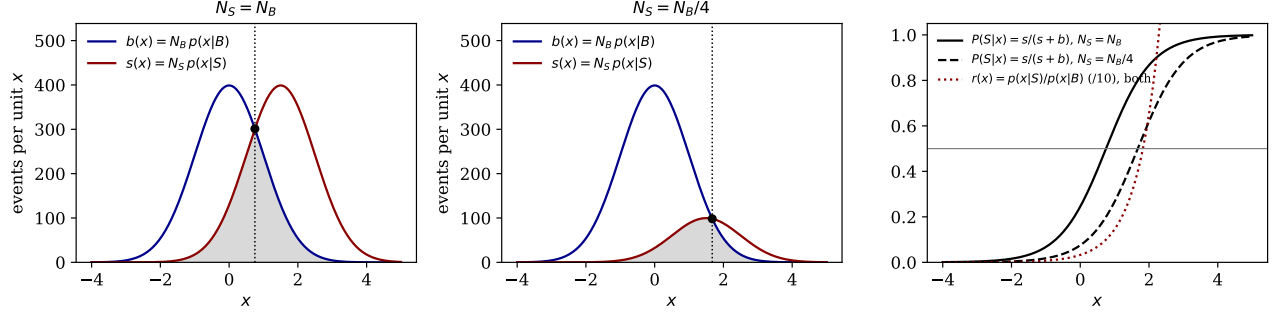


Figure 1: Left and center: expected event counts $s(x)$ and $b(x)$ for two overlapping Gaussian classes, for equal and for unequal class composition. At the crossing point $s = b$ the probability for signal is $1/2$. Right: $P(S|x) = s/(s+b)$ for both compositions, and the likelihood ratio $r(x)$, which is the same for both.

The distribution of the inputs $p(x_j)$ does not depend on θ , so like the evidence it does not move the maximum. What we have to model is the distribution of the target for a given input. An obvious choice, backed up by the central limit theorem, is the Gaussian likelihood: each target y_j is Gaussian distributed around the model prediction $f_\theta(x_j)$ with a width σ_j ,

$$p(y_j|x_j, \theta) = \frac{1}{\sqrt{2\pi}\sigma_j} \exp\left(-\frac{|y_j - f_\theta(x_j)|^2}{2\sigma_j^2}\right). \quad (4.3)$$

The loss is the negative logarithm of the likelihood,

$$\begin{aligned} \mathcal{L} &= -\log p(x_{\text{train}}|\theta) = -\sum_j \log p(y_j|x_j, \theta) - \sum_j \log p(x_j) \\ &= -\sum_j \log \left[\frac{1}{\sqrt{2\pi}\sigma_j} \exp\left(-\frac{|y_j - f_\theta(x_j)|^2}{2\sigma_j^2}\right) \right] + \text{const}(\theta) \\ &= \sum_j \left[\frac{|y_j - f_\theta(x_j)|^2}{2\sigma_j^2} + \log \sigma_j + \frac{1}{2} \log(2\pi) \right] + \text{const}(\theta) \\ &= \sum_j \frac{|y_j - f_\theta(x_j)|^2}{2\sigma_j^2} + \text{const}(\theta). \end{aligned} \quad (4.4)$$

The logarithm turns the product over training points into a sum, and the normalization terms do not depend on θ . When we do not know σ_j for each training point and take the same width σ for all N of them,

$$\mathcal{L} = \frac{1}{2\sigma^2} \sum_j |y_j - f_\theta(x_j)|^2 + \text{const}(\theta) = \frac{N}{2\sigma^2} \text{MSE} + \text{const}(\theta), \quad (4.5)$$

the MSE from Sec. 2, up to a positive factor and a constant, which both do not move the minimum. We will come back to learning σ on Day 4.

[adapted from Ref. [1], Sec. 1.2.2, *Fits and interpolations*]

5 Classification as a likelihood-ratio problem

We now turn to the standard problem of particle physics: is a given jet or event signal or background? Our training data are labelled pairs (x, y) with $y \in \{S, B\}$. Two different kinds of probabilities live on this setup, and we have to keep them apart:

- class densities $p(x|S)$ and $p(x|B)$, each normalized over phase space, $\int dx p(x|y) = 1$. They describe how signal and background events are distributed;

- label probabilities $P(S|x)$ and $P(B|x)$ at a given phase space point, normalized over the two labels, $P(S|x) + P(B|x) = 1$. They describe what we know about the label of one event.

We write densities in x with a lowercase p and probabilities of the discrete label with a capital P .² The priors $P(S)$ and $P(B)$ are the fractions of signal and background in the training data.

5.1 Signal probability and the optimal test statistic

We start with the simplest case in Fig. 1, one observable x . On the board we draw what a histogram shows, the expected number of signal and background events per unit x ,

$$s(x) = N_S p(x|S) \quad \text{and} \quad b(x) = N_B p(x|B) . \quad (5.1)$$

Because the two distributions overlap, no cut on x separates them perfectly. At the point where the two curves cross there are as many signal as background events, so the probability that an event there is signal is $1/2$. At any point x the answer is the familiar signal purity,

$$P(S|x) = \frac{s(x)}{s(x) + b(x)} . \quad (5.2)$$

With $N = N_S + N_B$ the priors are $P(S) = N_S/N$ and $P(B) = N_B/N$. Inserting s and b and dividing numerator and denominator by N gives Bayes' theorem,

$$P(S|x) = \frac{N_S p(x|S)}{N_S p(x|S) + N_B p(x|B)} = \frac{\frac{N_S}{N} p(x|S)}{\frac{N_S}{N} p(x|S) + \frac{N_B}{N} p(x|B)} = \frac{p(x|S) P(S)}{p(x|S) P(S) + p(x|B) P(B)} , \quad (5.3)$$

where the denominator is the density of the mixed sample, $p(x) = p(x|S)P(S) + p(x|B)P(B)$. Dividing numerator and denominator by $p(x|B) P(S)$ expresses it through the ratio of the class densities,

$$P(S|x) = \frac{r(x)}{r(x) + P(B)/P(S)} \quad \text{with} \quad r(x) = \frac{p(x|S)}{p(x|B)} . \quad (5.4)$$

Only for equal compositions, $N_S = N_B$, it becomes $p(x|S)/[p(x|S) + p(x|B)]$. The center panel of Fig. 1 shows why the composition matters: with fewer signal events the crossing point moves towards the signal peak, and $P(S|x)$ changes with it. The prior is recovered as the average of the signal probability over the full sample,

$$\int dx P(S|x) p(x) = \int dx \frac{p(x|S) P(S)}{p(x)} p(x) = P(S) \int dx p(x|S) = P(S) . \quad (5.5)$$

The ratio $r(x)$ does not depend on the composition. Seen as a function of the hypothesis, the class density $p(x|S)$ is the likelihood of the signal hypothesis for the observed x , and $r(x)$ is the ratio of the two likelihoods. The Neyman-Pearson lemma “says that the ratio of the two likelihoods is the most powerful test statistic to distinguish the two underlying hypotheses, defined as the smallest false negative error for a given false positive rate” [1, Sec. 2]. A selection is a cut $r(x) > r_{\text{cut}}$, so any strictly increasing function of $r(x)$ defines the same selections and the same ROC curve. According to Eq.(5.4), $P(S|x)$ is such a function, for any composition,

$$\frac{dP(S|x)}{dr} = \frac{(r + P(B)/P(S)) - r}{(r + P(B)/P(S))^2} = \frac{P(B)/P(S)}{(r + P(B)/P(S))^2} > 0 , \quad (5.6)$$

and unlike $r(x) \in [0, \infty)$ it lives in $[0, 1]$. In one dimension we can compute all of this by hand. For a realistic, high-dimensional phase space we cannot, and the question becomes how a network can learn $P(S|x)$ from samples.

²This deviates from Ref. [1], Sec. 2, which uses $p_{\text{data},S}$ both for the class density and for the label probability.

Worked example: two Gaussians. For the classes of Fig. 1, $p(x|B) = \mathcal{N}(x; 0, 1)$ and $p(x|S) = \mathcal{N}(x; \mu, 1)$ with $\mu = 1.5$, the normalizations cancel in the likelihood ratio,

$$r(x) = \frac{e^{-(x-\mu)^2/2}}{e^{-x^2/2}} = \exp\left(\frac{x^2 - (x-\mu)^2}{2}\right) = \exp\left(\mu x - \frac{\mu^2}{2}\right) \Rightarrow \log r(x) = \mu x - \frac{\mu^2}{2}, \quad (5.7)$$

linear in x . The curves cross where $s = b$, that is $N_S p(x|S) = N_B p(x|B)$ or $r(x) = N_B/N_S$,

$$\mu x_c - \frac{\mu^2}{2} = \log \frac{N_B}{N_S} \Rightarrow x_c = \frac{\mu}{2} + \frac{1}{\mu} \log \frac{N_B}{N_S} = \begin{cases} 0.75 & N_S = N_B \\ 0.75 + \frac{\log 4}{1.5} = 1.67 & N_S = N_B/4 \end{cases}, \quad (5.8)$$

the two crossing points marked in Fig. 1. Inserting $r(x)$ into Eq.(5.4),

$$P(S|x) = \frac{1}{1 + \frac{P(B)}{P(S)} \frac{1}{r(x)}} = \frac{1}{1 + \exp\left(-\mu x + \frac{\mu^2}{2} - \log \frac{P(S)}{P(B)}\right)} = \text{Sigmoid}\left(\mu x - \frac{\mu^2}{2} + \log \frac{P(S)}{P(B)}\right), \quad (5.9)$$

with the sigmoid function of Eq.(5.10) below. For two Gaussians of equal width the optimal classifier is exactly a sigmoid of a linear function of x , so a single linear layer with a sigmoid output is already optimal. The class composition only shifts the argument by $\log P(S)/P(B)$.

5.2 Cross entropy

[...] our classification network should encode and return a signal probability for a given jet or event, which means the final network layer has to ensure that the network output is $f_\theta(x) \in [0, 1]$. For usual networks this is not the case, but we can easily enforce this by replacing the ReLU activation function in the network output layer with a sigmoid function,

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}} \Leftrightarrow \text{Sigmoid}^{-1}(x) \equiv \text{Logit}(x) = \log \frac{x}{1-x}. \quad (5.10)$$

— taken from Ref. [1], Sec. 2, *Classification and feature spaces*

Writing the argument as z and the output as $C = \text{Sigmoid}(z)$, the inverse follows from solving $C = 1/(1 + e^{-z})$ for z ,

$$e^{-z} = \frac{1}{C} - 1 = \frac{1-C}{C} \Leftrightarrow z = \text{Sigmoid}^{-1}(C) \equiv \text{Logit}(C) = \log \frac{C}{1-C}, \quad (5.11)$$

and its derivative can be written in terms of the sigmoid itself,

$$\frac{d \text{Sigmoid}(z)}{dz} = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \frac{e^{-z}}{1 + e^{-z}} = \text{Sigmoid}(z) (1 - \text{Sigmoid}(z)). \quad (5.12)$$

We interpret the network output as the model for the label probability,

$$C_\theta(x) \equiv P_\theta(S|x) \quad \text{and} \quad 1 - C_\theta(x) = P_\theta(B|x). \quad (5.13)$$

The loss follows from exactly the same steps as the MSE in Sec. 4. The training data are now pairs (x_j, y_j) of an input and a label, and we look for the most probable parameters. With Bayes' theorem and a flat prior, Eq.(4.1), and the factorization of Eq.(4.2),

$$\hat{\theta} = \text{argmax}_\theta p(\theta|x_{\text{train}}) \simeq \text{argmax}_\theta p(x_{\text{train}}|\theta) = \text{argmax}_\theta \prod_j P_\theta(y_j|x_j) p(x_j) = \text{argmax}_\theta \prod_j P_\theta(y_j|x_j). \quad (5.14)$$

Again the inputs drop out, and we have to model the distribution of the target for a given input. The only difference to the regression is the target itself. For regression it was a continuous number, and we assumed a Gaussian distribution around $f_\theta(x)$. Now the target is a label with only two possible values. We encode it as a number,

$$y = 1 \quad \text{for signal} \quad \text{and} \quad y = 0 \quad \text{for background}. \quad (5.15)$$

A random variable with two outcomes is a (possibly biased) coin flip. If the probability for $y = 1$ is ρ , the probability for $y = 0$ has to be $1 - \rho$, and we can write both cases in one formula, the Bernoulli distribution

$$P_{\text{Bernoulli}}(y; \rho) = \rho^y (1 - \rho)^{1-y} = \begin{cases} \rho & y = 1 \\ 1 - \rho & y = 0 \end{cases} \quad \text{with} \quad \langle y \rangle = \rho, \quad \sigma^2 = \rho(1 - \rho). \quad (5.16)$$

The exponents only act as a switch that picks the right factor. Mean and variance follow from summing over the two outcomes,

$$\begin{aligned} \langle y \rangle &= 0 \cdot (1 - \rho) + 1 \cdot \rho = \rho, \\ \langle y^2 \rangle &= 0^2 \cdot (1 - \rho) + 1^2 \cdot \rho = \rho \quad \Rightarrow \quad \sigma^2 = \langle y^2 \rangle - \langle y \rangle^2 = \rho - \rho^2 = \rho(1 - \rho). \end{aligned} \quad (5.17)$$

The single parameter ρ is at the same time the probability for $y = 1$ and the mean of y , just like the Gaussian is fixed by its mean once we fix the width.

At a given phase space point x the label of an event is exactly such a coin flip. In the overlap region of Fig. 1 an event can be signal or background, and it is signal with probability $\rho = P(S|x)$. Our network models this probability, so the Bernoulli likelihood of one training event is

$$P_{\theta}(y_j|x_j) = P_{\text{Bernoulli}}(y_j; C_{\theta}(x_j)) = C_{\theta}(x_j)^{y_j} (1 - C_{\theta}(x_j))^{1-y_j}. \quad (5.18)$$

The loss is again the negative logarithm of the likelihood, which defines the binary cross entropy,

$$\begin{aligned} \mathcal{L}_{\text{class}} &= -\log p(x_{\text{train}}|\theta) = -\sum_j \log P_{\theta}(y_j|x_j) - \sum_j \log p(x_j) \\ &= -\sum_j \log \left[C_{\theta}(x_j)^{y_j} (1 - C_{\theta}(x_j))^{1-y_j} \right] + \text{const}(\theta) \\ &= -\sum_j \left[y_j \log C_{\theta}(x_j) + (1 - y_j) \log (1 - C_{\theta}(x_j)) \right] + \text{const}(\theta) \\ &= -\sum_{j \in S} \log C_{\theta}(x_j) - \sum_{j \in B} \log (1 - C_{\theta}(x_j)) + \text{const}(\theta), \end{aligned} \quad (5.19)$$

where in the last step $y_j = 1$ removes the second term for signal events and $y_j = 0$ the first term for background events.

[adapted from Ref. [1], Sec. 1.2.6, *Bayesian networks* and Sec. 2, *Classification and feature spaces*, in the notation of these notes]

Side by side, the two derivations differ only in the distribution of the target,

$$\begin{aligned} \text{MSE: } p(y|x, \theta) &= \mathcal{N}(y; f_{\theta}(x), \sigma) \quad \Rightarrow \quad -\log p = \frac{|y - f_{\theta}(x)|^2}{2\sigma^2} + \text{const} \\ \text{BCE: } P_{\theta}(y|x) &= C_{\theta}(x)^y (1 - C_{\theta}(x))^{1-y} \quad \Rightarrow \quad -\log P = -y \log C_{\theta}(x) - (1 - y) \log (1 - C_{\theta}(x)). \end{aligned} \quad (5.20)$$

We divide by the number of training events N , which does not move the minimum, and write each sum as an average over its own class,

$$\frac{1}{N} \sum_{j \in S} \log C_{\theta}(x_j) = \frac{N_S}{N} \frac{1}{N_S} \sum_{j \in S} \log C_{\theta}(x_j) \rightarrow P(S) \left\langle \log C_{\theta}(x) \right\rangle_{p(x|S)}, \quad (5.21)$$

and the same for the background. For a large training set the averages become expectation values over the class densities,

$$\boxed{\mathcal{L}_{\text{class}} = -P(S) \left\langle \log C_{\theta}(x) \right\rangle_{p(x|S)} - P(B) \left\langle \log (1 - C_{\theta}(x)) \right\rangle_{p(x|B)}}. \quad (5.22)$$

For the training we need the gradient. Writing the network output as $C_{\theta} = \text{Sigmoid}(z)$ with the last-layer output z , and using the sigmoid derivative, $d \log C / dz = 1 - C$ and $d \log (1 - C) / dz = -C$, the loss of one event has the simple derivative

$$\mathcal{L}_i = -y \log C - (1 - y) \log (1 - C) \quad \Rightarrow \quad \frac{\partial \mathcal{L}_i}{\partial z} = -y (1 - C) + (1 - y) C = C - y, \quad (5.23)$$

the difference between prediction and label, which back-propagation then passes through the network.

Equivalently, we can derive this loss from

[...] the Kullback–Leibler divergence, which compares two probability distributions, evaluated on a dataset corresponding to the first distribution,

$$D_{\text{KL}}[p_a, p_b] = \left\langle \log \frac{p_a}{p_b} \right\rangle_{p_a} \equiv \int dx p_a(x) \log \frac{p_a(x)}{p_b(x)}. \quad (5.24)$$

— taken from Ref. [1], Sec. 1.2.6, *Bayesian networks*

The KL-divergence is non-negative and vanishes only if the two distributions agree. The positivity follows from $\log z \leq z - 1$, with equality only for $z = 1$,

$$-D_{\text{KL}}[p_a, p_b] = \int dx p_a(x) \log \frac{p_b(x)}{p_a(x)} \leq \int dx p_a(x) \left(\frac{p_b(x)}{p_a(x)} - 1 \right) = \int dx p_b(x) - \int dx p_a(x) = 1 - 1 = 0. \quad (5.25)$$

We apply the KL-divergence to the true and learned label probabilities at each x , where the expectation value is a sum over the two labels, and average over phase space,

$$\begin{aligned} \left\langle D_{\text{KL}}[P(y|x), P_\theta(y|x)] \right\rangle_{p(x)} &= \left\langle \sum_{y=S,B} P(y|x) \log \frac{P(y|x)}{P_\theta(y|x)} \right\rangle_{p(x)} \\ &= \underbrace{\left\langle \sum_{y=S,B} P(y|x) \log P(y|x) \right\rangle_{p(x)}}_{\text{const}(\theta)} - \left\langle \sum_{y=S,B} P(y|x) \log P_\theta(y|x) \right\rangle_{p(x)}. \end{aligned} \quad (5.26)$$

With $p(x) P(y|x) = p(x|y) P(y)$ the second term becomes

$$\begin{aligned} - \int dx p(x) \sum_y P(y|x) \log P_\theta(y|x) &= - \sum_y P(y) \int dx p(x|y) \log P_\theta(y|x) \\ &= -P(S) \left\langle \log C_\theta(x) \right\rangle_{p(x|S)} - P(B) \left\langle \log (1 - C_\theta(x)) \right\rangle_{p(x|B)}, \end{aligned} \quad (5.27)$$

which is again Eq.(5.22). The ratio inside this KL-divergence compares *truth and model* for the same label probability. It is not the Neyman-Pearson ratio $r(x)$ in Eq.(5.4), which compares *signal and background* class densities.

[adapted from Ref. [1], Sec. 1.2.6, *Bayesian networks* and Sec. 2, *Classification and feature spaces*]

5.3 What the classifier learns

The KL-form tells us the answer immediately: the loss is minimal when $P_\theta(y|x) = P(y|x)$ everywhere. We can check this with a variation of the integrand of Eq.(5.22) with respect to C ,³

$$\begin{aligned} \frac{\delta}{\delta C} \left[P(S) p(x|S) \log C + P(B) p(x|B) \log(1 - C) \right] &= \frac{P(S) p(x|S)}{C} - \frac{P(B) p(x|B)}{1 - C} \stackrel{!}{=} 0 \\ \Leftrightarrow (1 - C) P(S) p(x|S) &= C P(B) p(x|B) \\ \Leftrightarrow P(S) p(x|S) &= C [P(S) p(x|S) + P(B) p(x|B)] \\ \Leftrightarrow C(x) &= \frac{P(S) p(x|S)}{P(S) p(x|S) + P(B) p(x|B)} = P(S|x). \end{aligned} \quad (5.28)$$

The second derivative of the bracket, $-P(S) p(x|S)/C^2 - P(B) p(x|B)/(1 - C)^2 < 0$, confirms a maximum of the bracket, so a minimum of the loss. We can optimize point by point, because the integrand is a sum of independent

³In Ref. [1], Sec. 2, the background term of this loss appears with a positive sign, $-\langle \log C \rangle_{p(x|S)} + \langle \log(1 - C) \rangle_{p(x|B)}$; the variation there uses the correct sign.

contributions from each x , weighted with positive densities. The trained classifier returns the label probability of Eq.(5.4), not the likelihood ratio itself. Inverting the relation, with $1 - C = P(B) p(x|B) / [P(S) p(x|S) + P(B) p(x|B)]$, gives the likelihood ratio,

$$\frac{C(x)}{1 - C(x)} = \frac{P(S)}{P(B)} \frac{p(x|S)}{p(x|B)}. \quad (5.29)$$

For balanced training data the prior factor is one. “This formula makes it explicit how the optimal classifier is related to the ratio of two likelihoods. Using it to extract a likelihood ratio is also called the **likelihood ratio trick**” [1, Sec. 2].

[adapted from Ref. [1], Sec. 2, *Classification and feature spaces*, with the class priors kept]

In the right panel of Fig. 1 we see that $C(x)$ and $r(x)$ are monotonically related, so cutting on the network output is the Neyman-Pearson optimal selection. We will use Eq.(5.29) again on Day 3, to reweight one sample into another.

Hands-on: we train small networks on the two-moons dataset with PyTorch. A network with a single ReLU node can only place one straight cut; a wider and deeper network learns a curved decision boundary. For both we plot $C/(1 - C)$ over the plane and compare it to the true likelihood ratio. The notebook is available at <https://github.com/SaschaDief/grad-days-ml-tutorials>.

6 Classification at the LHC

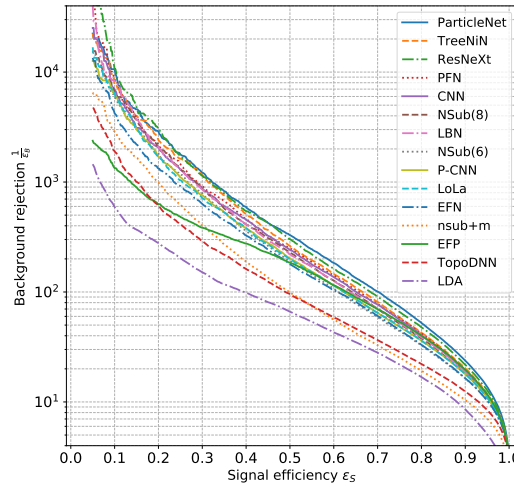


Figure 2: ROC curves for all top-tagging algorithms evaluated on the standard test sample. Figure from Ref. [2].

Neural network classifiers are the most established ML application in particle physics: jet tagging (top, b , quark vs gluon), event selection, triggers. The top-tagging comparison in Fig. 2 shows that architecture matters once the network sees low-level inputs, and that all of them are judged by the same ROC curve built from the learned likelihood ratio.

References

- [1] T. Plehn, A. Butter, B. Dillon, T. Heimel, C. Krause, and R. Winterhalder, *Modern Machine Learning for LHC Physicists*, [arXiv:2211.01421](https://arxiv.org/abs/2211.01421) [hep-ph].
- [2] G. Kasieczka, T. Plehn, *et al.*, *The Machine Learning landscape of top taggers*, SciPost Phys. **7** (2019) 014, [arXiv:1902.09914](https://arxiv.org/abs/1902.09914) [hep-ph].